Utilisation du kit NUCLEO-L432KC pour traitement du signal

Introduction aux filtres numériques

Généralités

Un filtre numérique est une fonction qui effectue une modification spectrale d'un signal échantillonné d'entrée « **x** » pour générer un signal de sortie « **y** » en utilisant un élément programmable (Microcontrôleur, FPGA, ordinateur ...)

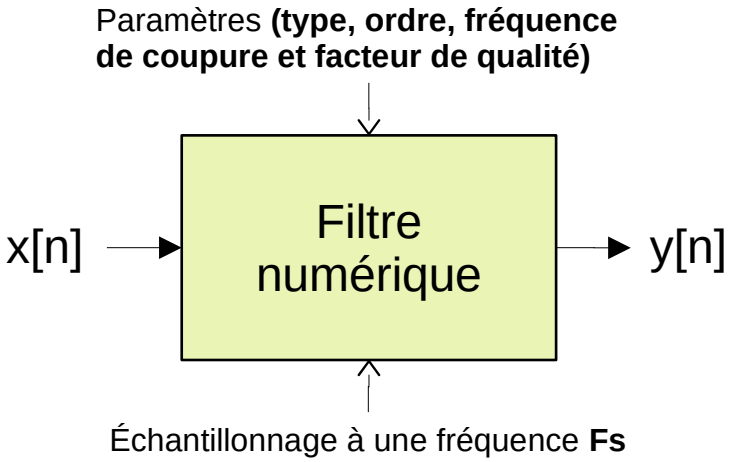


Figure 1 : Bloc fonctionnel d'un filtre numérique

Cela est obtenu par un algorithme qui calcule la valeur de l'échantillon présent de la sortie « **y[n]** » à partir des échantillons d'entrée présents « **x[n]** » et précédents « **x[n-k]** » (**k>0**), et/ou des échantillons de sortie précédents « **y[n-k]** ».

Ce calcul correspond à une équation de récurrence de la forme générale suivante :

$$y_{[n]} = \sum_{k=0}^{N} a_k \cdot x_{[n-k]} - \sum_{k=1}^{M} b_k \cdot y_{[n-k]}$$

N et **M** étant liés à l'ordre du filtre

Pour trouver l'équation de récurrence de ce filtre à partir de la fonction de transfert équivalente analogique, il faut utiliser un outil mathématique appelé « Transformée en Z ». On obtient ainsi une fonction de transfert de cette forme :

$$H_{(z)} = \frac{Y_{(z)}}{X_{(z)}} = \frac{\sum_{k=0}^{N} a_k \cdot z^{-k}}{1 + \sum_{k=1}^{M} b_k \cdot z^{-k}}$$

$\mathbf{z^{-k}}$ indique un retard de **k** échantillons

Cette transformée ne sera pas abordée dans ce TP car il existe une multitude d'outils logiciels permettant de faire les calculs.

Catégories de filtres

Selon les présences et les valeurs des coefficients $\mathbf{b_k}$ (appelés Pôles) et $\mathbf{a_k}$ (appelés Zéros)**,** différents type de filtres existent. En dehors de la nature du filtre (passe-haut, passe-bas, passe-bande …) et de son ordre, il y a deux catégories :

- les filtres à réponse impulsionnelle finie (**FIR**) qui possèdent une équation de récurrence sans coefficient de type $\mathbf{b_k}$ **(pour k>1)** mais dont le nombre de coefficients $\mathbf{a_k}$ est important selon l'ordre,
- les filtres à réponse impulsionnelle infinie (**IIR**) qui possèdent une équation de récurrence avec des coefficients $\mathbf{a_k}$ et $\mathbf{b_k}$ en nombre limité selon l'ordre.

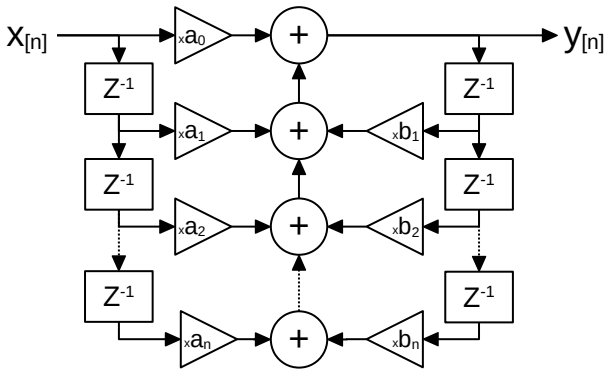


Figure 2 : Schéma bloc d'un filtre IIR

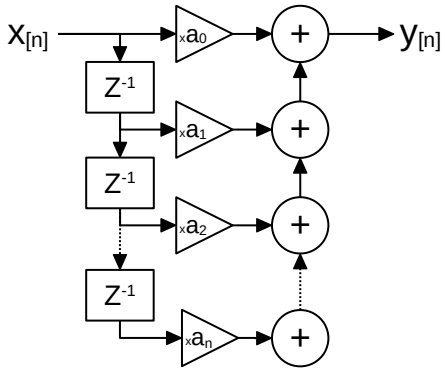


Figure 3 : schéma bloc d'un filtre FIR

Si ces derniers semblent plus performants, il y a cependant un problème de stabilité avec les filtres IIR.

Les filtres Biquad

Dans la mise en œuvre de filtres numériques du second ordre, le **Biquad** est intéressant car il met peu de coefficients en œuvre pour obtenir l'équivalent d'un **second ordre** analogique, ce qui nécessite moins de calculs.

Cependant c'est un filtre type **IIR** qui peut avoir des problèmes de stabilité.

Voici l'équation générique d'un **Biquad** :

$$y_{[n]} = a_0 \cdot x_{[n]} + a_1 \cdot x_{[n-1]} + a_2 \cdot x_{[n-2]} - b_1 \cdot y_{[n-1]} - b_2 \cdot y_{[n-2]}$$

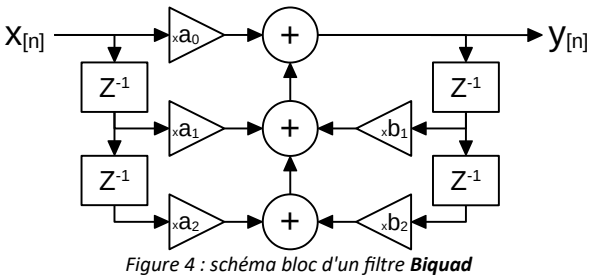


*Figure 4 : schéma bloc d'un filtre **Biquad***

Selon les valeurs des coefficients on peut obtenir différents types de filtres (passe haut, passe-bas …) , différentes fréquences de coupure et différents facteurs de qualité.

Voici les formules qui permettent de calculer ces coefficients pour un filtre passe-bas, un filtre passe-haut et un filtre passe bande.

- On calcule d'abord 2 coefficients généraux :

$$K = \tan\left(\frac{\pi \cdot F_C}{F_S}\right) \quad \rightarrow \quad norm = \frac{1}{1 + \frac{K}{Q} + K^2}$$

avec : $\mathbf{F_C}$ = fréquence de coupure, $\mathbf{F_S}$ = fréquence d'échantillonnage et **Q** = facteur de qualité

- Puis les coefficients finaux selon le type de filtre :

Coefficient	Passe bas	Passe-haut	Passe-bande
a_0	$K^2 \cdot norm$	$norm$	$\frac{K}{Q} \cdot norm$
a_1	$2 \cdot a_0$	$-2 \cdot a_0$	0
a_2	a_0	a_0	$-a_0$
b_1	$2 \cdot (K^2 - 1) \cdot norm$	$2 \cdot (K^2 - 1) \cdot norm$	$2 \cdot (K^2 - 1) \cdot norm$
b_2	$(1 - \frac{K}{Q} + K^2) \cdot norm$	$(1 - \frac{K}{Q} + K^2) \cdot norm$	$(1 - \frac{K}{Q} + K^2) \cdot norm$

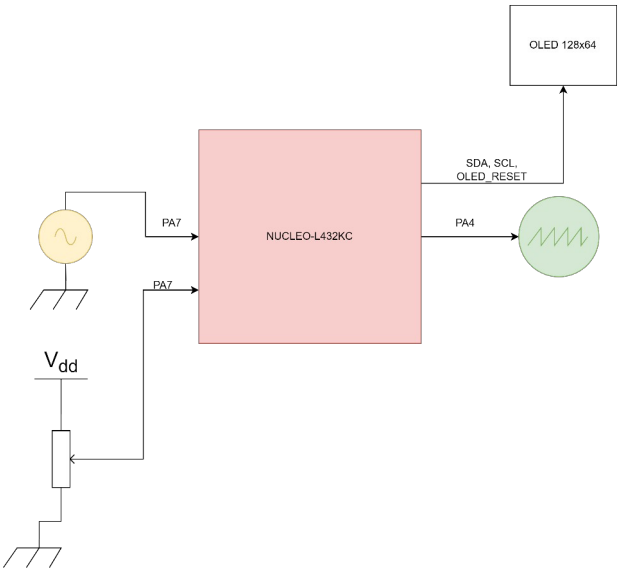
Attention : Dans le cas de calculs de ces coefficients par le programme du microcontrôleur, ceux-ci ne doivent pas être effectués par la routine d'interruption, ni en permanence dans la boucle sans-fin, sous peine de surcharger l'exécution du programme.

Ils existent des outils logiciels en ligne pour effectuer ces calculs :

- https://www.earlevel.com/main/2021/09/02/biquad-calculator-v3/
- https://www.earlevel.com/main/2011/01/02/biquad-formulas/
- https://www.earlevel.com/main/2003/02/28/biquads/

Description

Ce document présente un exemple de traitement du signal (mode DSP) avec un microcontrôleur STM32L432KC (kit NUCLEO).
On dispose de 2 entrées analogiques (ADC), la première pour convertir la tension continue d'un potentiomètre de réglage, la seconde pour l'échantillonnage d'un signal variable analogique.
La sortie analogique (DAC) permet de restituer le signal analogique après traitement.
Un afficheur OLED ainsi qu'une LED sont rajoutés pour la diffusion d'information et le débogage.
Voici le schéma fonctionnel correspondant :



Le filtre réalisé est un passe-bas du second ordre biquad avec une fréquence de coupure de 100Hz et un facteur de qualité de 0,707.
Voici les coefficients calculés :

$$y_{[n]} = a_0 \cdot x_{[n]} + a_1 \cdot x_{[n-1]} + a_2 \cdot x_{[n-2]} - b_1 \cdot y_{[n-1]} - b_2 \cdot y_{[n-2]}$$

a_0 = 0,00009506001696614666
a_1 = 0,0001901200339322933
a_2 = 0,00009506001696614666

b_1=-1,972233470205696
b_2 = 0.9726137102735608

Création d'un nouveau projet

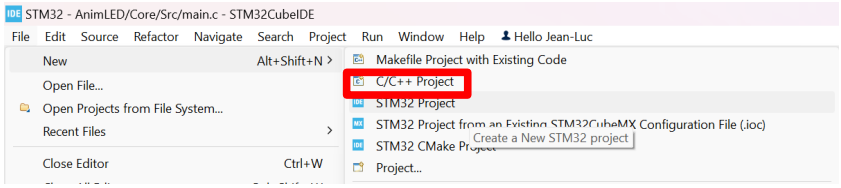


Figure 5: Sélection du projet pour STM32

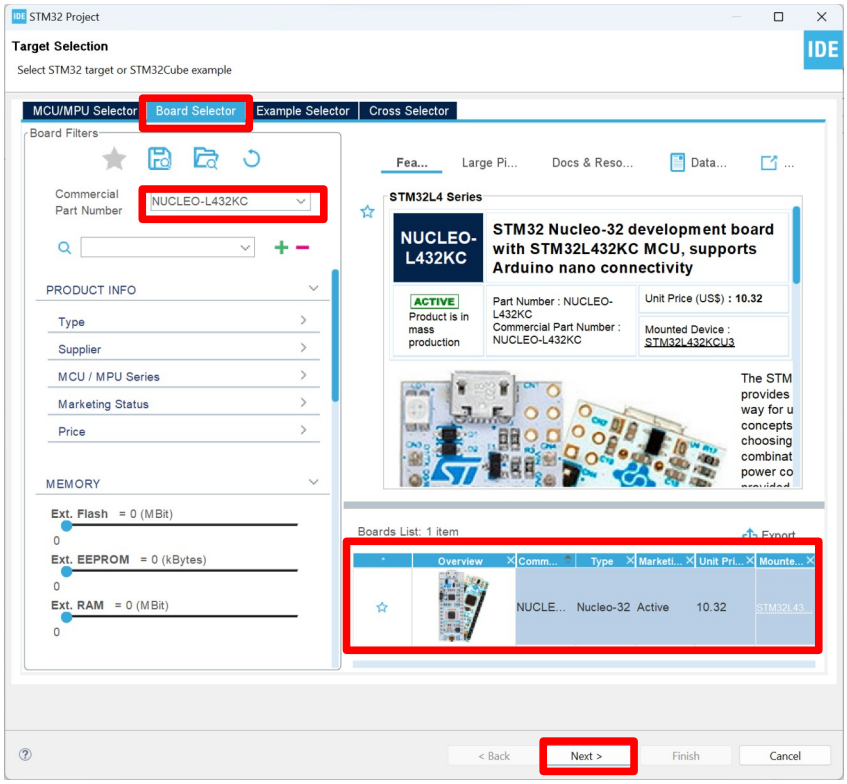


Figure 6: Choix de la cible

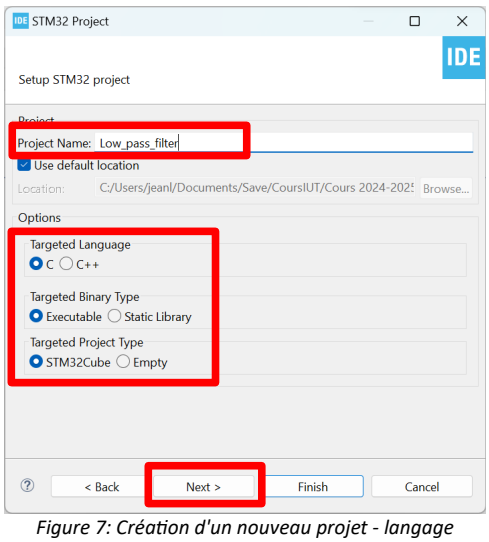


Figure 7: Création d'un nouveau projet - langage

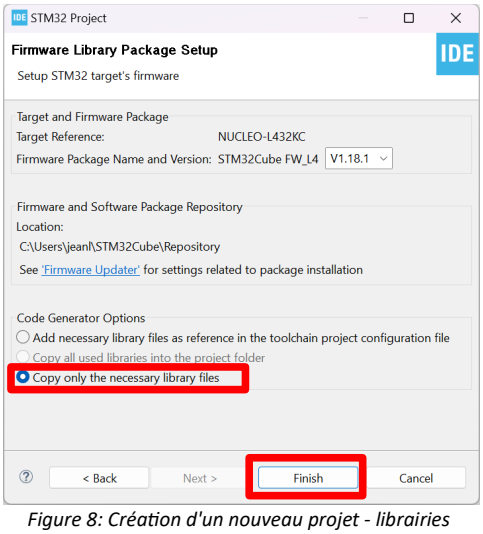


Figure 8: Création d'un nouveau projet - librairies

Configuration des périphériques

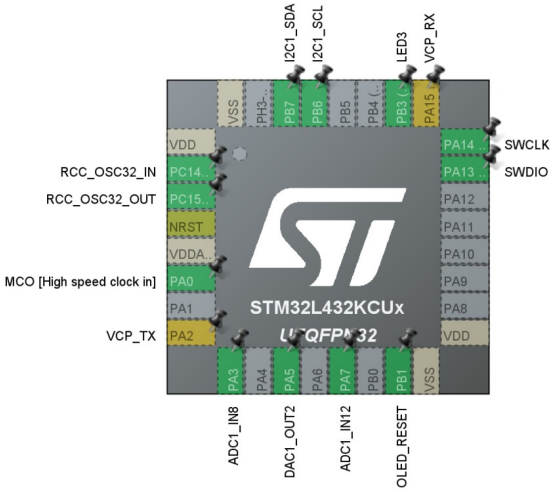


Figure 9: Configuration des GPIO en entrées / sorties analogiques

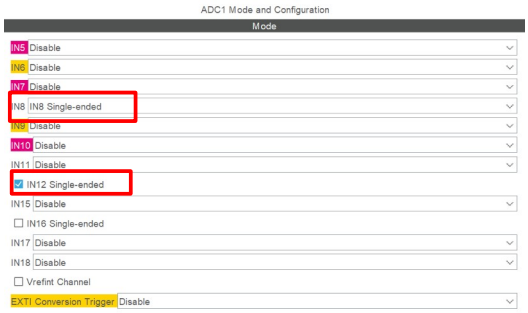


Figure 10: Activation des entrées de l'ADC

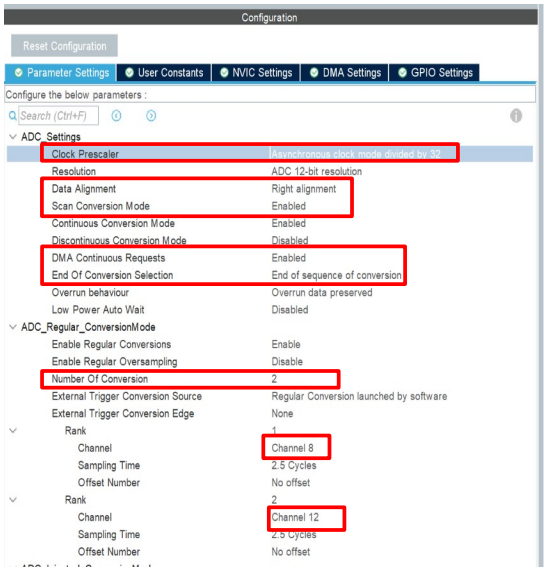


Figure 11: Configuration de l'ADC

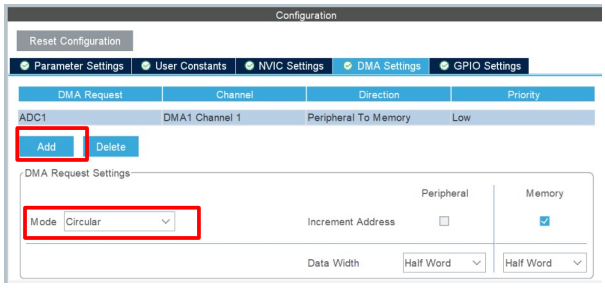


Figure 12: Configuration du mode DMA

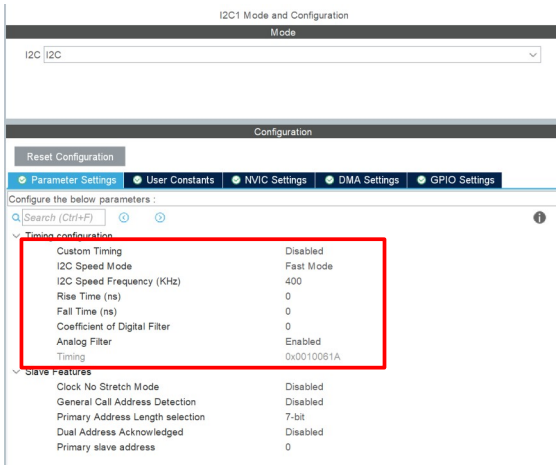


Figure 13: Configuration de l'I2C pour le OLED

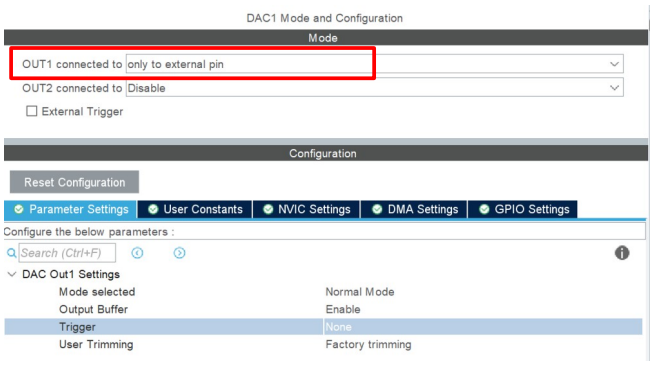


Figure 14: Configuration du DAC

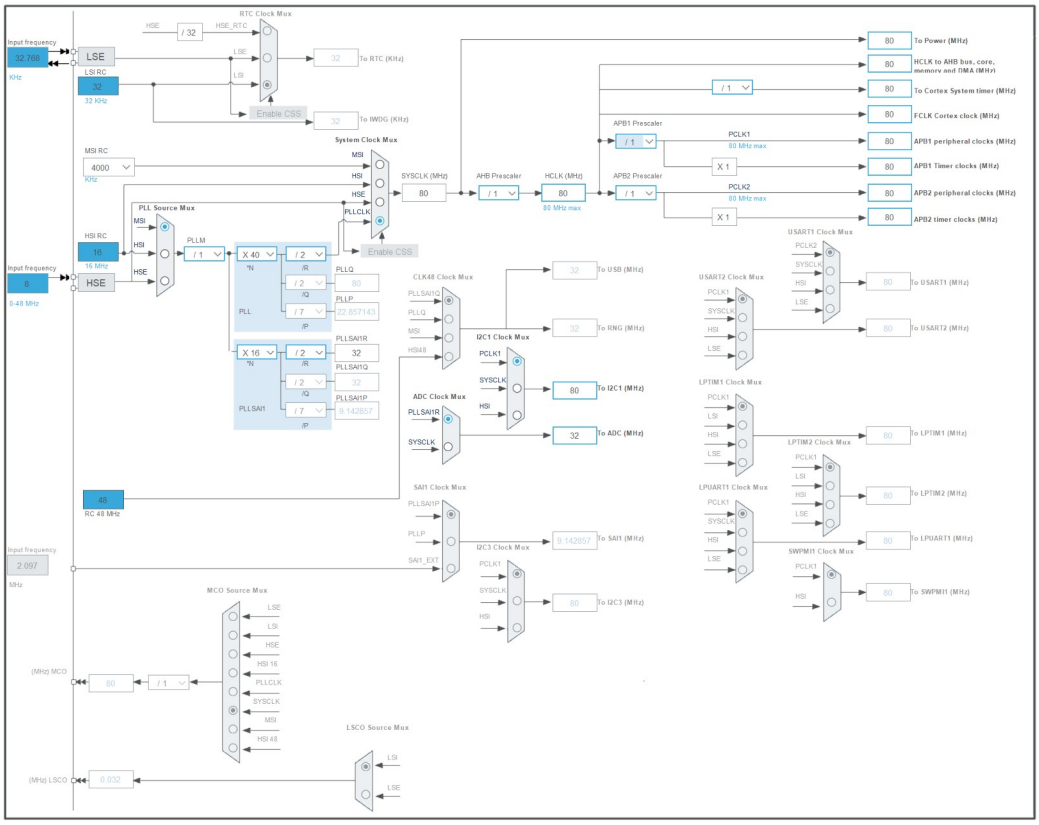


Figure 15: Configuration de l'horloge

Figure 16: Configuration du TIMER 6 pour interruption (échantillonnage à 32 kHZ)

TIM6 Mode and Configuration

Mode

☑ Activated

☐ One Pulse Mode

Configuration

Reset Configuration

Parameter Settings | User Constants | NVIC Settings | DMA Settings

Configure the below parameters :

Search (Ctrl+F)

Counter Settings	
Prescaler (PSC - 16 bits value)	0
Counter Mode	Up
Counter Period (AutoReload Register - 16 bits val...	2500
auto-reload preload	Disable
Trigger Output (TRGO) Parameters	
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

Prédivision par 1 pour garder une horloge à 80 MHz

Période de comptage à 2500 fois la période d'horloge. C'est à dire 32 kHz.

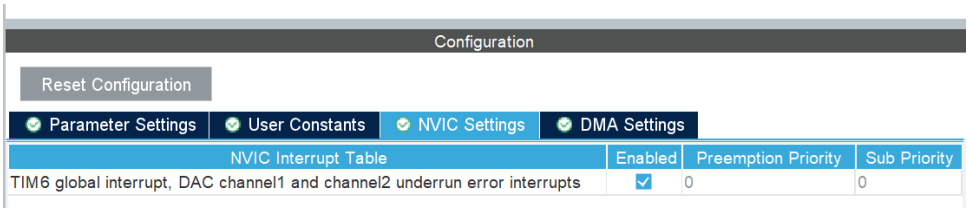


Figure 17: Activation de l'interruption du TIMER 6

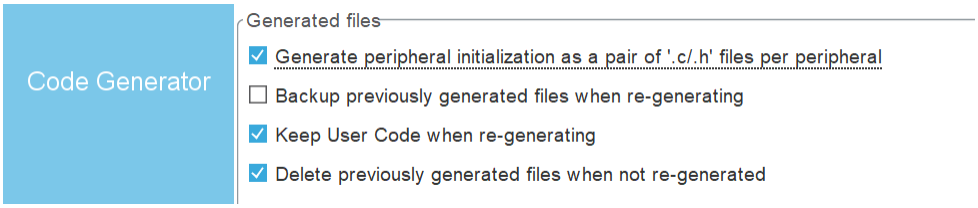


Figure 18: Générer des fichier séparés pour l'initialisation des périphériques

Codage en langage C dans le « main.c »

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2024 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "adc.h"
#include "dac.h"
#include "dma.h"
#include "i2c.h"
#include "tim.h"
#include "gpio.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "ssd1306.h"
#include <stdio.h>
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */
```

```c
/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */
uint16_t AdcIn[2];
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */
      char text[30];
  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_ADC1_Init();
  MX_DAC1_Init();
  MX_I2C1_Init();
  MX_TIM6_Init();
  /* USER CODE BEGIN 2 */
  HAL_ADC_Start_DMA(&hadc1, (uint32_t*)AdcIn, 2); // Start DMA ADC
  HAL_DAC_Start(&hdac1, DAC1_CHANNEL_1); //Start DAC1 Channel 2
  HAL_TIM_Base_Start_IT(&htim6);// Start TIMER 6
  //ECRAN OLED
```

```
  HAL_GPIO_WritePin(OLED_RESET_GPIO_Port,OLED_RESET_Pin,0); // Reset du OLED
  HAL_Delay(100);
  HAL_GPIO_WritePin(OLED_RESET_GPIO_Port,OLED_RESET_Pin,1);
  HAL_Delay(200);
  SSD1306_Init();
  SSD1306_GotoXY (0,0); // Coordonnées 0, 0
  SSD1306_Puts ("**LOW PASS FILTER**", &Font_7x10, SSD1306_COLOR_WHITE);
  SSD1306_UpdateScreen(); // update screen
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
        HAL_GPIO_TogglePin(LED3_GPIO_Port,LED3_Pin); // Clignotement LED pour
visualisation fonctionnement
        HAL_Delay(100);
        sprintf(text,"POT=%4d",AdcIn[0]); // Affichage de la position du potentiomètre

        SSD1306_GotoXY (0,20); // Coordonnées 0, 0
        SSD1306_Puts (text, &Font_7x10, SSD1306_COLOR_WHITE);
        SSD1306_UpdateScreen(); // update screen

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
      static double x0=0,x1=0,x2=0,y0,y1=0,y2=0;
      double a0 = 0.00009506001696614666;
      double a1 = 0.0001901200339322933;
      double a2 = 0.00009506001696614666;

      double b1 =-1.972233470205696;
      double b2 = 0.9726137102735608;

      if (htim==&htim6)
      {
            x0=AdcIn[1];
            y0=a0*x0+a1*x1+a2*x2-b1*y1-b2*y2;
            x2=x1;
            x1=x0;
            y2=y1;
            y1=y0;
            HAL_GPIO_TogglePin(SIGNAL_GPIO_Port, SIGNAL_Pin);
            HAL_DAC_SetValue(&hdac1, DAC1_CHANNEL_1, DAC_ALIGN_12B_R,(uint16_t)y0);

      }
}
/* USER CODE END 4 */
```